Ficheros en Java: chuleta I · Escribir

Acceso a Datos · 2º DAM · `java.nio.file` (`Files`, `Path`, `StandardOpenOption`)

Opciones de `Files.writeString` / `Files.write`

Quiero...	Opciones
Crear o **sobrescribir** todo (vaciar antes)	sin opciones, o `CREATE` + `TRUNCATE_EXISTING`
Añadir al final y crear si no existe	`CREATE` + `APPEND`
Añadir al final, el fichero **debe existir**	`APPEND`
Crear solo si **no existe** (falla si existe)	`CREATE_NEW`

TRAMPAS COMPROBADAS EJECUTANDO EL CÓDIGO

- `CREATE` **solo**, sobre un fichero que ya existe: NO lo vacía. Escribe encima desde el principio y deja restos. Ejemplo: se escribe `LARGO` `LARGO`, luego `corto`, y queda `corto` + `LARGO` debajo.
- `APPEND` **solo**, sobre un fichero que no existe: lanza `NoSuchFileException`. Falta `CREATE`.
- Sin ninguna opción equivale a `CREATE` + `TRUNCATE_EXISTING` + `WRITE`.
- Nunca combinar `APPEND` con `TRUNCATE_EXISTING`.

Código

```
Path ruta = Path.of("src/paquete/datos.txt");

// sobrescribir
Files.writeString(ruta, texto);

// añadir al final (crea si no existe)
Files.writeString(ruta, linea + "\n",
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);

// crear solo si no existe
Files.writeString(ruta, texto, StandardOpenOption.CREATE_NEW);
```

Escribir muchas líneas: acumular y escribir UNA vez

```
StringBuilder sb = new StringBuilder();
for (Alumno a : lista) {
    sb.append(a.getNombre()).append(";").append(a.getNota()).append("\n");
}
Files.writeString(ruta, sb);      // fuera del bucle
```

Si llamas a `writeString` dentro del bucle con `TRUNCATE_EXISTING`, cada vuelta borra lo anterior y solo queda la última línea.

Dónde falló en los ejercicios de clase

- **videojuegosAlquiler** y **alquilerInstrumentosMusicales**: opción "crear" con `CREATE` solo (restos) y opción "añadir" con `APPEND` solo (excepción si no existe el fichero).
- **listaAlumnos/Main**: opción 2 con `CREATE` solo.
- **alquilerInstrumentosMusicales**, opción 1: reescribe los ficheros una vez por instrumento, dentro del bucle.

Excepciones: todos estos métodos lanzan `IOException`. O `throws IOException` en el método, o `try/catch`.

Ficheros en Java: chuleta II · Leer, validar y Scanner

Acceso a Datos · 2º DAM

Leer

```
String todo = Files.readString(ruta);            // todo en un String
List<String> lineas = Files.readAllLines(ruta); // un String por línea
for (String l : lineas) { ... }

try (Scanner sc = new Scanner(new File(ruta))) { // se cierra solo
    while (sc.hasNextLine()) {
        String linea = sc.nextLine().trim();
    }
}
```

Procesar una línea `nombre;nota`

```
String[] partes = linea.split(";");
if (partes.length != 2) { /* línea malformada */ continue; }
try {
    String nombre = partes[0].trim();
    int marca = Integer.parseInt(partes[1].trim());
} catch (NumberFormatException e) { /* dato no numérico */ }
```

Orden de validaciones: 1) línea vacía, 2) número de campos tras `split`, 3) conversión a número con `try/catch`, 4) rango (negativos).

Errores típicos con `Scanner(System.in)`

Fallo	Arreglo
`nextLine()` devuelve vacío tras `nextInt()`	Poner `sc.nextLine()` después de `nextInt()` / `nextDouble()`
Varios `Scanner(System.in)`	Uno solo, `static` o pasado por parámetro
`"Suspenso"` ≠ `"SUSPENSO"`	`equalsIgnoreCase`, o `.trim().toUpperCase()`
Enum mal escrito rompe el programa	`valueOf(...)` dentro de `try/catch (IllegalArgumentException)`
Letras donde iba un número	Capturar `InputMismatchException` y descartar con `sc.nextLine()`
`default` del menú lanza excepción	Mostrar "Opción no válida" y volver al menú

Formato

Quiero	Uso
entero	`%d` (no `d%`)
decimal, 2 cifras	`%.2f`
texto	`%s`
salto de línea	`%n` (no `/n`)

`Files` / `Path`

```
Files.exists(ruta)
Files.notExists(ruta)
Files.createDirectories(dir)
Files.deleteIfExists(ruta)
Files.size(ruta)
ruta.getFileName()
```

Plantilla de ejercicio

```
public static void main(String[] args) throws IOException {
    Path ruta = Path.of("src/paquete/datos.txt");
    // 1. leer y validar cada línea
    // 2. calcular con colecciones: HashMap<String, ArrayList<Integer>>
    // 3. escribir el resultado (elegir bien las opciones)
}
```